

Projet personnel — Développement & DevOps

Recette API

Conception, développement et déploiement d'une API REST
sécurisée avec pipeline CI/CD complet et interface web

Auteur	Manoach HOSSOU DODO
Dépôt backend	https://github.com/hdmanoach/recette-api
Dépôt frontend	https://github.com/hdmanoach/recette-frontend
Démo API	https://recette-api-l2r6.onrender.com/docs
Démo interface	https://recette-frontend-five.vercel.app

Réalisé en 2026

Document généré avec L^AT_EX

Table des matières

1	Introduction et contexte	3
1.1	Origine du projet	3
1.2	Objectifs	3
1.3	Vue d'ensemble du résultat	3
2	Architecture générale	4
2.1	Vue d'ensemble	4
2.2	Flux de communication	4
2.3	Séparation des deux dépôts	4
3	Backend — API FastAPI	5
3.1	Choix technologiques	5
3.2	Structure du projet	5
3.3	Modélisation des données	6
3.3.1	Table des recettes	6
3.3.2	Validation avec Pydantic	6
3.4	Authentification et sécurité	6
3.4.1	Inscription et hachage des mots de passe	6
3.4.2	Connexion et génération du jeton JWT	7
3.4.3	Protection des routes	7
3.4.4	Contrôle de propriété	7
3.4.5	Protection contre les attaques par force brute	7
3.5	Journalisation (logs)	8
3.6	Endpoints de l'API	8
3.7	Gestion de la configuration	8
3.8	Bascule automatique SQLite / PostgreSQL	8
3.9	Endpoint de supervision (<i>health check</i>)	9
4	Tests, qualité et intégration continue	10
4.1	Tests automatisés	10
4.2	Analyse statique du code	10
4.3	Pipeline GitHub Actions	10
4.4	Conteneurisation avec Docker	11
5	Frontend — Interface Next.js	12
5.1	Choix technologiques	12
5.2	Pages principales	12
5.3	Gestion de l'authentification côté client	12
5.4	Communication avec l'API	12

6	Déploiement en production	14
6.1	Backend sur Render, base de données sur Supabase	14
6.2	Frontend sur Vercel	14
6.3	Configuration CORS	14
7	Difficultés rencontrées et solutions	16
8	Compétences mobilisées et acquises	17
8.1	Développement backend	17
8.2	Développement frontend	17
8.3	Qualité logicielle et DevOps	17
9	Pistes d'amélioration	18
10	Conclusion	19

Chapitre 1

Introduction et contexte

1.1 Origine du projet

Ce projet est né d'un dépôt GitHub nommé initialement `github-actions-demo`, créé dans le but d'apprendre à mettre en place un pipeline d'intégration et de déploiement continu (CI/CD) avec GitHub Actions. Plutôt que de partir d'un exemple abstrait, le choix a été fait de construire une véritable application de bout en bout : une API de gestion de recettes de cuisine, accompagnée d'une interface web, afin que chaque notion apprise (tests automatisés, linting, conteneurisation, déploiement) s'applique à un cas concret et cohérent.

Le projet a été développé de façon progressive et pédagogique : chaque fonctionnalité a été ajoutée étape par étape, testée, corrigée, puis intégrée au pipeline de qualité avant de passer à la suivante.

1.2 Objectifs

- Concevoir une API REST complète avec FastAPI (Python).
- Mettre en place une base de données relationnelle persistante.
- Sécuriser l'API avec un système d'authentification par jetons JWT.
- Automatiser la qualité du code (tests, linting) via GitHub Actions.
- Conteneuriser l'application avec Docker.
- Déployer l'API et une interface web en production, avec déploiement continu.
- Documenter l'ensemble de la démarche et de l'architecture.

1.3 Vue d'ensemble du résultat

Le projet final se compose de deux applications distinctes, déployées séparément et communiquant via HTTP :

- un **backend** FastAPI, exposant une API REST, déployé sur **Render**, connecté à une base de données PostgreSQL managée par **Supabase** ;
- un **frontend** Next.js, consommant cette API, déployé sur **Vercel**.

Résultat concret : une application fonctionnelle, accessible publiquement, avec authentification, gestion de contenu par utilisateur, et une chaîne d'intégration continue qui valide automatiquement chaque changement de code avant qu'il n'atteigne la production.

Chapitre 2

Architecture générale

2.1 Vue d'ensemble

L'architecture suit un modèle client-serveur classique, avec une séparation nette entre le frontend (interface utilisateur) et le backend (logique métier et données).

Frontend	Backend	Base de données
Next.js (React, TypeScript) Déployé sur Vercel Requêtes HTTP (fetch)	FastAPI (Python) Déployé sur Render API REST + JWT	PostgreSQL (Supabase, production) SQLite (développement) Gérée via SQLAlchemy (ORM)

2.2 Flux de communication

1. L'utilisateur charge l'interface Next.js dans son navigateur.
2. Le frontend effectue des requêtes HTTP (`fetch`) vers l'API, à l'adresse définie par la variable d'environnement `NEXT_PUBLIC_API_URL`.
3. L'API FastAPI traite la requête : validation des données (Pydantic), vérification d'authentification si nécessaire (JWT), interrogation de la base de données via SQLAlchemy.
4. La réponse JSON est renvoyée au frontend, qui met à jour l'affichage.

Pour autoriser cette communication entre deux origines différentes (`localhost:3000` en développement, `vercel.app` en production, vers `onrender.com`), le middleware **CORS** de FastAPI a été configuré explicitement.

2.3 Séparation des deux dépôts

Le choix a été fait d'utiliser **deux dépôts Git distincts** plutôt qu'un seul monorepo :

- **recette-api** : le backend, avec son propre pipeline CI/CD et son propre déploiement (Render).
- **recette-frontend** : le frontend, avec son propre déploiement (Vercel).

Cette séparation reflète une pratique courante en entreprise : chaque application a son propre cycle de vie, ses propres tests, et peut être déployée indépendamment de l'autre.

Chapitre 3

Backend — API FastAPI

3.1 Choix technologiques

Technologie	Rôle
FastAPI	Framework web Python, génération automatique de la documentation OpenAPI/Swagger
SQLAlchemy	ORM (Object-Relational Mapping), abstraction de la base de données
Pydantic	Validation des données entrantes et sortantes de l'API
PostgreSQL (Supabase) / SQLite	Base de données relationnelle (production / développement)
python-jose	Génération et vérification des jetons JWT
passlib + bcrypt	Hachage sécurisé des mots de passe
pytest	Tests automatisés
ruff	Analyse statique et style du code (linting)
Docker	Conteneurisation de l'application

3.2 Structure du projet

Le code a été progressivement réorganisé, passant d'un unique fichier `main.py` monolithique à une architecture modulaire basée sur les **routeurs** FastAPI :

```
recette-api/
|-- main.py                # Point d'entree : app, CORS, middleware, routers
|-- database.py            # Connexion a la base de donnees
|-- models.py              # Modeles Pydantic (validation) et SQLAlchemy (tables)
|-- security.py            # JWT, hachage de mot de passe, anti-bruteforce
|-- middleware.py          # Journalisation (logs) de chaque requete
|-- routers/
|   |-- auth.py            # /register, /login
|   |-- recipes.py         # CRUD des recettes
|   '-- logs.py            # /logs
|-- test_main.py           # Tests automatisés (pytest)
|-- requirements.txt
|-- pyproject.toml         # Configuration de ruff
|-- Dockerfile
'-- .github/workflows/ci.yml # Pipeline CI/CD
```

Ce découpage sépare clairement les responsabilités : la sécurité (`security.py`), l'observabilité (`middleware.py`), et chaque domaine métier dans son propre routeur.

3.3 Modélisation des données

Deux familles de modèles sont utilisées côté backend, une distinction essentielle à comprendre :

- les modèles **Pydantic** (`Recipe`, `UserCreate`, `UserLogin`, `UserOut`) valident les données JSON échangées avec les clients de l'API ;
- les modèles **SQLAlchemy** (`RecipeDB`, `UserDB`, `LogEntry`) décrivent les tables réelles de la base de données.

3.3.1 Table des recettes

```
class RecipeDB(Base):
    __tablename__ = "recipes"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    title = Column(String, nullable=False)
    ingredients = Column(JSON, nullable=False)
    instructions = Column(JSON, nullable=False)
    prep_time = Column(Integer, nullable=False)
    servings = Column(Integer, nullable=False)
    owner_id = Column(Integer, ForeignKey("users.id"))
```

Le champ `owner_id` établit une clé étrangère vers la table des utilisateurs : chaque recette appartient à un unique propriétaire.

3.3.2 Validation avec Pydantic

La classe `Recipe` impose des règles métier avant même que la donnée n'atteigne la base :

```
class Recipe(BaseModel):
    id: int | None = None
    title: str = Field(..., min_length=1)
    ingredients: list[str] = Field(..., min_length=1)
    instructions: list[str] = Field(..., min_length=1)
    prep_time: int = Field(..., gt=0)
    servings: int = Field(..., ge=1, le=20)
```

Toute requête ne respectant pas ces contraintes (titre vide, temps de préparation négatif, etc.) est automatiquement rejetée par FastAPI avec un code d'erreur 422, sans qu'aucune logique métier n'ait à être écrite manuellement.

3.4 Authentification et sécurité

3.4.1 Inscription et hachage des mots de passe

Les mots de passe ne sont jamais stockés en clair. La bibliothèque `passlib`, avec l'algorithme `bcrypt`, calcule une empreinte irréversible :

```
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)
```

3.4.2 Connexion et génération du jeton JWT

Lors de la connexion, un jeton **JWT** (JSON Web Token) signé est généré et renvoyé au client. Ce jeton contient l'identité de l'utilisateur (**sub**) et une date d'expiration, sans nécessiter de stockage de session côté serveur :

```
def create_access_token(data: dict) -> str:
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) + timedelta(
        minutes=ACCESS_TOKEN_EXPIRE_MINUTES
    )
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
```

3.4.3 Protection des routes

Une dépendance FastAPI (`get_current_user`) decode et vérifie le jeton présenté dans l'en-tête `Authorization: Bearer <token>`, puis retrouve l'utilisateur correspondant en base. Elle est injectée dans les endpoints qui nécessitent une authentification :

```
@router.post("/recipes")
def create_recipe(
    recipe: Recipe,
    db: Session = Depends(get_db),
    current_user: UserDB = Depends(get_current_user),
):
    ...
```

3.4.4 Contrôle de propriété

Au-delà de la simple authentification, une vérification d'**autorisation** garantit qu'un utilisateur ne peut modifier ou supprimer que ses propres recettes :

```
if recipe.owner_id != current_user.id:
    raise HTTPException(
        status_code=403,
        detail="Vous n'etes pas autorise a supprimer cette recette",
    )
```

3.4.5 Protection contre les attaques par force brute

Un compteur en mémoire enregistre les tentatives de connexion échouées par nom d'utilisateur. Après cinq échecs en quinze minutes, le compte est temporairement verrouillé (HTTP 429) :

```
MAX_LOGIN_ATTEMPTS = 5
LOCKOUT_DURATION_MINUTES = 15

def is_locked_out(username: str) -> bool:
    attempts = failed_login_attempts.get(username, [])
    now = datetime.now(timezone.utc)
    recent = [a for a in attempts
               if now - a < timedelta(minutes=LOCKOUT_DURATION_MINUTES)]
    failed_login_attempts[username] = recent
    return len(recent) >= MAX_LOGIN_ATTEMPTS
```


3.5 Journalisation (logs)

Un **middleware** intercepte chaque requête HTTP reçue par l'API et enregistre en base : méthode, chemin, code de statut, adresse IP, et nom d'utilisateur (extrait du jeton JWT si présent). Ces journaux sont consultables via l'endpoint protégé **GET /logs**, avec pagination et tri par date décroissante.

3.6 Endpoints de l'API

Méthode	Route	Description
GET	/	Message de bienvenue
GET	/health	Vérification de l'état de l'API et de la connexion à la base
POST	/register	Créer un compte utilisateur
POST	/login	Connexion, obtention d'un jeton JWT
GET	/recipes	Lister les recettes (filtres, pagination)
GET	/recipes/{id}	Récupérer une recette
POST	/recipes	Créer une recette (<i>authentifié</i>)
POST	/recipes/bulk	Créer plusieurs recettes (<i>authentifié</i>)
PUT	/recipes/{id}	Modifier une recette (<i>propriétaire</i>)
DELETE	/recipes/{id}	Supprimer une recette (<i>propriétaire</i>)
GET	/logs	Consulter le journal des requêtes (<i>authentifié</i>)

L'endpoint **GET /recipes** accepte plusieurs paramètres de requête optionnels : **skip** et **limit** pour la pagination, **title** et **ingredient** pour une recherche textuelle partielle (insensible à la casse), et **max_prep_time** pour filtrer par temps de préparation maximal.

3.7 Gestion de la configuration

Les informations sensibles (clé secrète JWT, URL de base de données, origines CORS autorisées) ne sont jamais codées en dur dans le code source. Elles sont chargées depuis des variables d'environnement via **python-dotenv**, avec un fichier **.env** local exclu du dépôt Git :

```
load_dotenv()
SECRET_KEY = os.getenv("SECRET_KEY")
ALGORITHM = os.getenv("ALGORITHM", "HS256")
ACCESS_TOKEN_EXPIRE_MINUTES = int(os.getenv("ACCESS_TOKEN_EXPIRE_MINUTES", "30"))
```

En production (Render) et dans le pipeline CI (GitHub Actions), ces mêmes variables sont injectées via les systèmes de secrets propres à chaque plateforme.

3.8 Bascule automatique SQLite / PostgreSQL

Pour permettre un développement local simple sans dépendance externe, tout en garantissant une persistance réelle en production, **database.py** choisit dynamiquement le moteur de base de données :

```
DATABASE_URL = os.getenv("DATABASE_URL")

if DATABASE_URL:
    if DATABASE_URL.startswith("postgres://"):
        DATABASE_URL = DATABASE_URL.replace("postgres://", "postgresql://", 1)
    engine = create_engine(DATABASE_URL)
```

```
else:
    os.makedirs("data", exist_ok=True)
    engine = create_engine(
        "sqlite:///./data/recettes.db",
        connect_args={"check_same_thread": False},
    )
```

Si la variable `DATABASE_URL` est définie (cas de la production), PostgreSQL est utilisé. Sinon, l'application retombe sur SQLite, stocké localement dans un fichier.

Point d'attention réseau : Render (offre gratuite) ne supporte pas IPv6, alors que l'URI de connexion *directe* fournie par Supabase résout en IPv6. Il faut donc impérativement utiliser l'URI de **Connection Pooling (Session)** de Supabase, sur le port 6543 et non le port direct 5432, pour que la connexion aboutisse depuis Render.

3.9 Endpoint de supervision (*health check*)

Un endpoint `GET /health` a été ajouté pour deux raisons :

- vérifier à tout moment que l'API répond et que la connexion à la base de données fonctionne (utile pour un diagnostic rapide) ;
- maintenir la base **Supabase** active, les projets gratuits Supabase pouvant être mis en pause après une période d'inactivité prolongée.

Un service externe gratuit (cron-job.org) appelle automatiquement cet endpoint toutes les 72 heures, ce qui suffit à maintenir le projet Supabase actif sans intervention manuelle.

Chapitre 4

Tests, qualité et intégration continue

4.1 Tests automatisés

Les tests, écrits avec `pytest` et le `TestClient` de FastAPI, utilisent une base de données **SQLite en mémoire**, entièrement isolée de la base de développement ou de production. Un point technique important a dû être résolu : par défaut, chaque connexion SQLite en mémoire crée une base distincte. La solution consiste à forcer une connexion unique et partagée via `StaticPool` :

```
engine = create_engine(  
    "sqlite:///memory:",  
    connect_args={"check_same_thread": False},  
    poolclass=StaticPool,  
)
```

La dépendance `get_db` de l'application est remplacée (`dependency_overrides`) par une version pointant vers cette base de test. Une fixture `pytest` crée les tables avant chaque test et les détruit après, garantissant une isolation complète entre les tests.

4.2 Analyse statique du code

L'outil `ruff` est exécuté systématiquement pour vérifier le style et détecter les erreurs courantes (imports inutilisés, imports mal triés, variables mal utilisées). Une exception a été explicitement configurée dans `pyproject.toml` pour la règle B008, qui signale par erreur l'usage standard et recommandé de `Depends(...)` dans les signatures de fonctions FastAPI :

```
[tool.ruff.lint]  
ignore = ["B008"]
```

4.3 Pipeline GitHub Actions

Le pipeline CI/CD se déclenche automatiquement à chaque `push` ou `pull request` sur la branche principale. Il exécute deux jobs en parallèle :

- **test** : installation des dépendances, vérification du style (`ruff`), exécution des tests (`pytest`) ;
- **docker-build** : construction de l'image Docker, pour garantir que l'application reste conteneurisable à chaque changement.

```
jobs:  
  test:  
    runs-on: ubuntu-latest
```

```

steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-python@v5
    with:
      python-version: "3.12"
  - run: pip install -r requirements.txt
  - run: ruff check .
  - run: pytest
  env:
    SECRET_KEY: ${ secrets.SECRET_KEY }

docker-build:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - run: docker build -t recette-api .

```

La clé secrète nécessaire aux tests d'authentification est injectée depuis les *secrets* du dépôt GitHub, sans jamais apparaître en clair dans le code ni dans les journaux d'exécution.

4.4 Conteneurisation avec Docker

L'image Docker est construite en optimisant la mise en cache des couches : les dépendances sont installées avant que le reste du code source ne soit copié, de sorte qu'une modification du code seul ne force pas une réinstallation complète des paquets Python.

```

FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD uvicorn main:app --host 0.0.0.0 --port ${PORT:-8000}

```

La persistance des données en local (hors production) est assurée par un **volume Docker**, qui fait correspondre le dossier `data/` du conteneur à un dossier du système hôte :

```
docker run -p 8000:8000 -v $(pwd)/data:/app/data recette-api
```

Chapitre 5

Frontend — Interface Next.js

5.1 Choix technologiques

Le frontend a été développé avec **Next.js** (App Router), **TypeScript** et **Tailwind CSS**, en cohérence avec les standards actuels de l'écosystème React.

5.2 Pages principales

Route	Rôle
/	Liste des recettes (accès public), actions de modification si connecté
/login	Formulaire de connexion, stockage du jeton JWT
/register	Formulaire d'inscription
/recipes/new	Création d'une recette (authentifié)
/recipes/[id]/edit	Modification d'une recette existante (route dynamique)

5.3 Gestion de l'authentification côté client

Après une connexion réussie, le jeton JWT renvoyé par l'API est stocké dans le `localStorage` du navigateur, puis réutilisé dans l'en-tête `Authorization` de chaque requête nécessitant une authentification :

```
const token = localStorage.getItem("token");

await fetch(`${process.env.NEXT_PUBLIC_API_URL}/recipes`, {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    Authorization: `Bearer ${token}`,
  },
  body: JSON.stringify(recipeData),
});
```

L'état de connexion de l'interface (affichage conditionnel des boutons « Modifier » / « Supprimer », bascule entre « Se connecter » et « Se déconnecter ») est dérivé de la présence de ce jeton.

5.4 Communication avec l'API

L'adresse de l'API est injectée via une variable d'environnement publique :

```
NEXT_PUBLIC_API_URL=https://recette-api-l2r6.onrender.com
```

Le préfixe `NEXT_PUBLIC_` est requis par Next.js pour qu'une variable soit accessible côté navigateur, et non uniquement côté serveur.

Chapitre 6

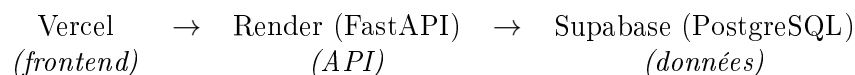
Déploiement en production

6.1 Backend sur Render, base de données sur Supabase

Le backend est déployé sur **Render**, directement à partir du `Dockerfile` du dépôt. Chaque `push` sur la branche principale déclenche un redéploiement automatique.

La persistance des données est assurée par une base **PostgreSQL** **managée par Supabase**, choisie après un premier essai avec l'offre PostgreSQL gratuite de Render elle-même : celle-ci expire au bout de quelques semaines sur le plan gratuit, alors que Supabase propose une offre gratuite permanente (sous réserve d'une activité régulière, d'où l'endpoint `/health` associé à un cron externe décrit au chapitre précédent).

L'architecture de production peut se résumer ainsi :



auquel s'ajoute **cron-job.org**, qui interroge `/health` toutes les 72 heures pour maintenir le projet Supabase actif.

Les variables d'environnement de production sont configurées directement dans le tableau de bord Render, séparément du code source :

Variable	Valeur
DATABASE_URL	URI de <i>connection pooling</i> (session) Supabase, port 6543
SECRET_KEY	Clé secrète de signature des jetons JWT
ALGORITHM	HS256
ACCESS_TOKEN_EXPIRE_MINUTES	30
CORS_ORIGINS	Origines autorisées (frontend local et déployé)

6.2 Frontend sur Vercel

Le frontend est déployé sur **Vercel**, la plateforme des créateurs de Next.js, avec détection automatique du framework et déploiement continu à chaque `push`.

6.3 Configuration CORS

Pour autoriser les requêtes du frontend déployé vers l'API, les origines autorisées sont définies dynamiquement à partir d'une variable d'environnement, plutôt que codées en dur :

```
cors_origins_raw = os.getenv(
    "CORS_ORIGINS", "http://localhost:3000"
```

```
)  
origins = [o.strip() for o in cors_origins_raw.split(",")]  
  
app.add_middleware(  
    CORSMiddleware,  
    allow_origins=origins,  
    allow_credentials=True,  
    allow_methods=["*"],  
    allow_headers=["*"],  
)
```


Chapitre 7

Difficultés rencontrées et solutions

Cette section retrace les principaux obstacles techniques rencontrés durant le développement, et la façon dont ils ont été résolus — une partie volontairement conservée car elle documente un apprentissage réel, utile en cas de projet similaire futur.

Problème	Cause et solution
Incompatibilité <code>passlib</code> / <code>bcrypt</code>	Les versions récentes de <code>bcrypt</code> ont retiré un attribut interne attendu par <code>passlib</code> . Solution : fixer la version <code>bcrypt==4.0.1</code> .
Erreur 422 sur le formulaire d'autorisation Swagger	<code>OAuth2PasswordBearer</code> attend des données de formulaire, alors que <code>/login</code> recevait du JSON. Solution : remplacement par <code>HTTPBearer</code> , plus adapté à un jeton collé directement.
Tables introuvables en tests (<code>no such table</code>)	Chaque connexion SQLite en mémoire crée une base séparée. Solution : usage de <code>StaticPool</code> pour forcer une connexion unique partagée.
Erreur CORS trompeuse en production	Une erreur serveur (500) survenant avant l'ajout des en-têtes CORS peut apparaître comme une erreur CORS dans le navigateur, masquant la vraie cause. Solution : consultation systématique des journaux serveur (Render) pour identifier l'erreur réelle.
<code>integer out of range</code> sur PostgreSQL	Un identifiant généré côté client avec <code>Date.now()</code> (timestamp en millisecondes) dépassait la capacité d'un entier PostgreSQL standard, sans poser problème sur SQLite (plus permissif). Solution : laisser la base de données générer les identifiants automatiquement (<code>autoincrement</code>), retrait de la génération manuelle côté client.
Perte de connexion PostgreSQL en local	L'URL interne fournie par Render n'est résolvable que depuis le réseau privé de Render. Solution : ne définir <code>DATABASE_URL</code> que dans les variables d'environnement de production, jamais en local.
Expiration de la base gratuite Render	L'offre PostgreSQL gratuite de Render est limitée dans le temps (quelques semaines). Solution : migration vers Supabase, dont l'offre gratuite est permanente sous réserve d'activité régulière du projet.
Échec de connexion Render → Supabase	L'URI de connexion <i>directe</i> de Supabase résout en IPv6, non supporté par le plan gratuit de Render. Solution : utilisation de l'URI de <i>connection pooling</i> (port 6543), compatible IPv4.

Chapitre 8

Compétences mobilisées et acquises

8.1 Développement backend

- Conception d'une API REST avec FastAPI (routage, validation, dépendances).
- Modélisation de données relationnelles avec SQLAlchemy (ORM).
- Authentification par jetons JWT et hachage sécurisé de mots de passe.
- Gestion des autorisations (propriété des ressources).

8.2 Développement frontend

- Construction d'une interface avec Next.js (App Router) et TypeScript.
- Gestion d'état React (`useState`, `useEffect`) et routes dynamiques.
- Consommation d'une API REST authentifiée depuis le client.

8.3 Qualité logicielle et DevOps

- Écriture de tests automatisés isolés (pytest).
- Analyse statique de code (ruff).
- Conteneurisation avec Docker (image, volumes, bonnes pratiques de cache).
- Construction d'un pipeline CI/CD avec GitHub Actions (jobs parallèles, secrets).
- Déploiement continu sur des plateformes cloud (Render, Vercel).
- Gestion de configuration par variables d'environnement et secrets.

Chapitre 9

Pistes d'amélioration

- **Rôles utilisateurs** : introduire un rôle administrateur avec des droits élargis (accès aux logs de tous les utilisateurs, modération).
- **Rafraîchissement de jeton** : ajouter un mécanisme de *refresh token* pour prolonger une session sans nouvelle connexion complète.
- **Catégorisation des recettes** : ajout d'un champ catégorie (entrée, plat, dessert) avec filtre dédié.
- **Upload d'images** : associer une photo à chaque recette, avec stockage externe (S3 ou équivalent).
- **Tests frontend** : ajouter des tests automatisés côté interface (Jest, Testing Library, ou Playwright pour des tests de bout en bout).
- **Observabilité renforcée** : export des journaux vers un service dédié plutôt qu'un stockage en base applicative.
- **Cookies httpOnly** : remplacer le stockage du jeton en `localStorage` par des cookies sécurisés, plus robustes face aux attaques de type XSS.

Chapitre 10

Conclusion

Ce projet, parti d'un simple dépôt de démonstration pour GitHub Actions, a permis de construire une application complète et fonctionnelle, couvrant l'ensemble de la chaîne de développement logicielle moderne : conception d'une API sécurisée, persistance des données, interface utilisateur, automatisation des tests et du déploiement.

Au-delà du résultat technique, la démarche progressive adoptée — une fonctionnalité à la fois, systématiquement testée avant de passer à la suivante — a permis de consolider une compréhension solide de chaque brique de l'architecture, plutôt qu'un assemblage superficiel de technologies. Les difficultés rencontrées en cours de route, documentées au chapitre 6, constituent une base de référence utile pour d'éventuels projets futurs de structure similaire.
